

Лекция по JavaScript (Часть 3)

Продолжительность: 3 часа

Предыдущие темы:

1. Введение в JavaScript: синтаксис, переменные, типы данных, операторы.
 2. Управляющие конструкции: условия (`if` , `else` , `switch`), циклы (`for` , `while` , `do...while`), функции.
-

План лекции:

1. Обзор предыдущих тем (15 минут)

- Краткое повторение: переменные, типы данных, операторы, условия, циклы, функции.
- Вопросы и ответы по пройденному материалу.

2. Массивы и методы работы с ними (45 минут)

- Что такое массивы?
- Создание массивов.
- Основные методы: `push` , `pop` , `shift` , `unshift` , `slice` , `splice` , `concat` , `indexOf` , `includes` .
- Итерация по массивам: `for` , `for...of` , `forEach` , `map` , `filter` , `reduce` .
- Практические примеры.

3. Объекты в JavaScript (45 минут)

- Что такое объекты?
- Создание объектов: литералы и конструкторы.
- Свойства и методы объектов.
- Доступ к свойствам: точечная нотация и квадратные скобки.
- Итерация по объектам: `for...in` .
- Практические примеры.

4. Работа с DOM (1 час)

- Что такое DOM?
- Поиск элементов: `getElementById` , `getElementsByClassName` , `getElementsByTagName` , `querySelector` , `querySelectorAll` .
- Изменение содержимого: `innerHTML` , `textContent` .
- Работа с атрибутами: `getAttribute` , `setAttribute` , `removeAttribute` .

- События: `addEventListener`, `removeEventListener`.
- Практические примеры: создание интерактивных элементов.

5. Перерыв (10 минут)

6. Асинхронность в JavaScript (45 минут)

- Что такое асинхронность?
- `setTimeout` и `setInterval`.
- Колбэки и проблемы "ада колбэков".
- Промисы (`Promise`): создание, обработка, цепочки вызовов.
- `async/await` : синтаксический сахар для промисов.
- Практические примеры.

7. Заключение и вопросы (20 минут)

- Подведение итогов.
- Ответы на вопросы.
- Рекомендации по дальнейшему изучению.

Подробное содержание лекции:

1. Обзор предыдущих тем (15 минут)

- Повторение ключевых моментов:
 - Переменные: `let`, `const`, `var`.
 - Давайте подробно разберём, как работают переменные в JavaScript, и чем отличаются `let`, `const` и `var`.

1. Общее о переменных

Переменные в JavaScript используются для хранения данных. Они могут содержать числа, строки, объекты, функции и другие типы данных. В JavaScript есть три способа объявления переменных: `let`, `const` и `var`.

2. `let`

Особенности:

- **Блочная область видимости:** Переменная, объявленная с помощью `let`, видна только внутри блока `{}`, в котором она объявлена.
- **Можно изменять:** Значение переменной, объявленной через `let`, можно изменять.
- **Нельзя переобъявлять:** В пределах одной области видимости переменную `let` нельзя объявить дважды.

Пример:

```
let x = 10;
if (true) {
  let x = 20; // Это другая переменная x, которая существует только
              // внутри этого блока
  console.log(x); // 20
}
console.log(x); // 10 (вне блока if)
```

Область видимости:

```
{
  let y = 5;
  console.log(y); // 5
}
console.log(y); // Ошибка: y is not defined (y не определена вне блока)
```

Изменение значения:

```
let z = 1;
z = 2; // Можно изменить
console.log(z); // 2
```

Переобъявление:

```
let a = 10;
let a = 20; // Ошибка: Identifier 'a' has already been declared
```

3. const

Особенности:

- **Блочная область видимости:** Как и `let`, переменная `const` видна только внутри блока `{ }`, в котором она объявлена.
- **Нельзя изменять:** Значение переменной, объявленной через `const`, нельзя изменить после инициализации.
- **Нельзя переобъявлять:** В пределах одной области видимости переменную `const` нельзя объявить дважды.
- **Обязательная инициализация:** Переменная `const` должна быть сразу инициализирована значением.

Пример:

```
const PI = 3.14;
console.log(PI); // 3.14

PI = 3.14159; // Ошибка: Assignment to constant variable
```

Область видимости:

```
{
  const NAME = "Alice";
  console.log(NAME); // Alice
}
console.log(NAME); // Ошибка: NAME is not defined
```

Объекты и массивы:

- Если переменная `const` содержит объект или массив, то сам объект или массив можно изменять, но нельзя переприсвоить переменную.

```
const person = { name: "John" };
person.name = "Alice"; // Можно изменить свойство объекта
console.log(person); // { name: "Alice" }

person = { name: "Bob" }; // Ошибка: Assignment to constant variable
```

4. `var`

Особенности:

- **Функциональная область видимости:** Переменная, объявленная с помощью `var`, видна внутри функции, в которой она объявлена, или глобально, если объявлена вне функции.
- **Можно изменять:** Значение переменной, объявленной через `var`, можно изменять.
- **Можно переобъявлять:** В пределах одной области видимости переменную `var` можно объявить несколько раз.
- **Поднятие (hoisting):** Переменные `var` "поднимаются" в начало своей области видимости, но их значение остаётся `undefined` до момента инициализации.

Пример:

```
var x = 10;
if (true) {
    var x = 20; // Это та же самая переменная x
    console.log(x); // 20
}
console.log(x); // 20 (переменная x изменена внутри блока if)
```

Область видимости:

```
function test() {
    var y = 5;
    console.log(y); // 5
}
console.log(y); // Ошибка: y is not defined (y не определена вне функции)
```

Поднятие (hoisting):

```
console.log(z); // undefined (переменная z объявлена, но не инициализирована)
var z = 10;
console.log(z); // 10
```

Переобъявление:

```
var a = 10;
var a = 20; // Можно переобъявить
console.log(a); // 20
```

5. Сравнение `let`, `const` и `var`

Характеристика	<code>let</code>	<code>const</code>	<code>var</code>
Область видимости	Блочная	Блочная	Функциональная
Изменяемость	Можно изменять	Нельзя изменять	Можно изменять
Переобъявление	Нельзя	Нельзя	Можно
Поднятие (hoisting)	Нет (но есть TDZ)	Нет (но есть TDZ)	Да
Инициализация	Необязательна	Обязательна	Необязательна

6. Когда что использовать?

- `const` : Используйте по умолчанию для всех переменных, значение которых не должно изменяться.
 - `let` : Используйте, если значение переменной будет изменяться.
 - `var` : Старайтесь избегать, так как его поведение может привести к ошибкам из-за функциональной области видимости и поднятия.
-

7. Примеры использования

Использование `const` :

```
const API_KEY = "12345"; // Константа, которая не изменяется
const user = { name: "John" }; // Объект, свойства которого можно
изменять
user.name = "Alice"; // Можно
user = {}; // Ошибка
```

Использование `let` :

```
let count = 0; // Переменная, которая будет изменяться
count = 1; // Можно
```

Использование `var` (не рекомендуется):

```
var oldVar = 10; // Устаревший способ
oldVar = 20; // Можно
```

8. Практическое задание

1. Объявите переменную с помощью `const` и попробуйте изменить её значение. Что произойдёт?
2. Объявите переменную с помощью `let` внутри блока `{ }` и попробуйте обратиться к ней вне блока. Что произойдёт?
3. Объявите переменную с помощью `var` внутри функции и попробуйте обратиться к ней вне функции. Что произойдёт?
4. Создайте объект с помощью `const` и измените его свойства. Что произойдёт?

Пример:

```
// 1. const
const age = 25;
age = 30; // Ошибка

// 2. let
{
    let temp = 10;
}
console.log(temp); // Ошибка

// 3. var
function test() {
    var temp = 5;
}
console.log(temp); // Ошибка

// 4. const с объектом
const car = { brand: "Toyota" };
car.brand = "Honda"; // Можно
car = {}; // Ошибка
```

Итог:

- Используйте `const` по умолчанию.

- Используйте `let` , если значение переменной будет изменяться.
 - Избегайте `var` , если это возможно.
 - Понимание областей видимости и поднятия поможет избежать ошибок.
-

- Типы данных: примитивы и объекты.

- **1. Типы данных: примитивы и объекты**

Теория:

В JavaScript данные делятся на два основных типа: **примитивы** и **объекты**.

Примитивы:

- Это простые, неизменяемые (immutable) значения.
- К примитивам относятся:
 - `number` (числа, включая `NaN` и `Infinity`),
 - `string` (строки),
 - `boolean` (`true` / `false`),
 - `null` (отсутствие значения),
 - `undefined` (значение не определено),
 - `symbol` (уникальные идентификаторы, добавленные в ES6).

Объекты:

- Это сложные, изменяемые (mutable) структуры данных.
- К объектам относятся:
 - Объекты (`{}`),
 - Массивы (`[]`),
 - Функции,
 - Даты,
 - Регулярные выражения и другие встроенные объекты.

Особенности:

- Прimitives передаются по значению (копируются).
- Объекты передаются по ссылке (изменение объекта в одной переменной влияет на все ссылки).

Практика:

```
// Прimitives

let num = 10; // number

let str = "Hello"; // string

let bool = true; // boolean

let empty = null; // null

let undef; // undefined

let sym = Symbol("id"); // symbol

// Объекты

let obj = { name: "John", age: 30 }; // объект

let arr = [1, 2, 3]; // массив

let func = function() { console.log("I'm a function!"); }; // функция

// Пример передачи по значению (прimitives)

let a = 5;

let b = a; // копирование значения

b = 10;

console.log(a); // 5 (значение не изменилось)

// Пример передачи по ссылке (объекты)

let obj1 = { name: "Alice" };
```

```
let obj2 = obj1; // копирование ссылки

obj2.name = "Bob";

console.log(obj1.name); // Bob (значение изменилось)
```

- Условия: `if`, `else`, `switch`.
- #####2. Условия: `if`, `else`, `switch`

Теория:

Условия позволяют выполнять код в зависимости от выполнения определённых условий.

`if` и `else`:

- `if` проверяет условие, и если оно истинно (`true`), выполняется блок кода.
- `else` выполняется, если условие в `if` ложно (`false`).
- Можно использовать `else if` для проверки нескольких условий.

`switch`:

- Используется для сравнения значения с несколькими вариантами.
- Каждый вариант обозначается ключевым словом `case`.
- Если ни один `case` не подходит, выполняется блок `default`.

Практика:

```
// if, else if, else

let age = 18;

if (age < 18) {

    console.log("You are a minor.");

} else if (age === 18) {
```

```
        console.log("You just became an adult!");
    } else {
        console.log("You are an adult.");
    }

    // switch

    let day = "Monday";

    switch (day) {

        case "Monday":

            console.log("Start of the work week.");

            break;

        case "Friday":

            console.log("End of the work week.");

            break;

        default:

            console.log("It's a regular day.");

    }
```

-
- Циклы: `for`, `while`, `do...while`.

- **3. Циклы: `for`, `while`, `do...while`**

Теория:

Циклы позволяют повторять выполнение кода несколько раз.

for :

- Используется, когда известно количество итераций.
- Состоит из трёх частей: инициализация, условие, шаг.

while :

- Используется, когда количество итераций неизвестно.
- Выполняется, пока условие истинно.

do...while :

- Похож на `while`, но гарантирует выполнение хотя бы одной итерации.

Практика:

```
// for

for (let i = 0; i < 5; i++) {

    console.log("Iteration:", i);

}

// while

let i = 0;

while (i < 5) {

    console.log("Iteration:", i);

    i++;

}

// do...while

let j = 0;

do {

    console.log("Iteration:", j);
```

```
j++;  
  
} while (j < 5);
```

- Функции: объявление, вызов, возврат значений.

• 4. Функции: объявление, вызов, возврат значений

Теория:

Функции — это блоки кода, которые можно вызывать многократно. Они помогают организовать код и избежать дублирования.

Объявление функции:

- Функция может быть объявлена с помощью ключевого слова `function`.
- Также можно использовать стрелочные функции (введены в ES6).

Вызов функции:

- Функция выполняется, когда её вызывают по имени.

Возврат значений:

- Функция может возвращать значение с помощью ключевого слова `return`.
- Если `return` отсутствует, функция возвращает `undefined`.

Практика:

```
// Объявление функции  
  
function greet(name) {  
  
    return "Hello, " + name + "!";  
  
}  
  
// Вызов функции
```

```
let message = greet("Alice");

console.log(message); // Hello, Alice!

// Стрелочная функция

const add = (a, b) => a + b;

console.log(add(2, 3)); // 5

// Функция без return

function noReturn() {

    console.log("This function does not return anything.");

}

let result = noReturn();

console.log(result); // undefined
```

Итог:

- **Типы данных:** Примитивы и объекты — основа работы с данными в JavaScript.
- **Условия:** Позволяют управлять потоком выполнения программы.
- **Циклы:** Позволяют повторять выполнение кода.
- **Функции:** Помогают организовать код и избежать дублирования.

Практическое задание:

1. Создайте переменные разных типов данных (примитивы и объекты) и выведите их в консоль.
2. Напишите условие, которое проверяет, является ли число положительным, отрицательным или нулём.
3. Используйте цикл `for`, чтобы вывести числа от 1 до 10.
4. Создайте функцию, которая принимает два числа и возвращает их сумму.

2. Массивы и методы работы с ними (45 минут)

- Массивы — это упорядоченные коллекции данных.
- Создание массива:

```
let arr = [1, 2, 3];  
let arr2 = new Array(1, 2, 3);
```

- Основные методы:
 - `push` — добавление элемента в конец.
 - `pop` — удаление элемента с конца.
 - `shift` — удаление элемента с начала.
 - `unshift` — добавление элемента в начало.
 - `slice` — создание нового массива из части исходного.
 - `splice` — изменение массива (удаление, замена, добавление элементов).
 - `concat` — объединение массивов.
 - `indexOf` — поиск индекса элемента.
 - `includes` — проверка наличия элемента.
- Итерация по массивам:

```
for (let i = 0; i < arr.length; i++) {  
  console.log(arr[i]);  
}  
  
arr.forEach((item, index) => {  
  console.log(item, index);  
});  
  
let newArr = arr.map(item => item * 2);  
let filteredArr = arr.filter(item => item > 2);  
let sum = arr.reduce((acc, item) => acc + item, 0);
```

Давайте подробно разберём каждый из этих примеров кода. Все они связаны с работой с массивами в JavaScript.

1. Цикл `for` для перебора массива

```
for (let i = 0; i < arr.length; i++) {  
    console.log(arr[i]);  
}
```

Комментарий:

- `for` — это классический цикл, который используется для перебора элементов массива.
- `let i = 0` — инициализация переменной-счётчика `i`, которая начинается с 0 (индексы массивов в JavaScript начинаются с 0).
- `i < arr.length` — условие выполнения цикла. Цикл будет выполняться, пока `i` меньше длины массива `arr`.
- `i++` — шаг цикла. После каждой итерации значение `i` увеличивается на 1.
- `arr[i]` — обращение к элементу массива по индексу `i`.
- `console.log(arr[i])` — вывод текущего элемента массива в консоль.

Пример:

```
let arr = [1, 2, 3, 4, 5];  
for (let i = 0; i < arr.length; i++) {  
    console.log(arr[i]);  
}  
// Вывод:  
// 1  
// 2  
// 3  
// 4  
// 5
```

2. Метод `forEach` для перебора массива

```
arr.forEach((item, index) => {  
    console.log(item, index);  
});
```

Комментарий:

- `forEach` — это метод массива, который позволяет перебрать все его элементы.

- `(item, index) => { ... }` — это стрелочная функция, которая принимает два аргумента:
 - `item` — текущий элемент массива.
 - `index` — индекс текущего элемента.
- `console.log(item, index)` — вывод текущего элемента и его индекса в консоль.

Пример:

```
let arr = [1, 2, 3, 4, 5];
arr.forEach((item, index) => {
    console.log(item, index);
});
// Вывод:
// 1 0
// 2 1
// 3 2
// 4 3
// 5 4
```

3. Метод `map` для создания нового массива

```
let newArr = arr.map(item => item * 2);
```

Комментарий:

- `map` — это метод массива, который создаёт новый массив, применяя функцию к каждому элементу исходного массива.
- `item => item * 2` — стрелочная функция, которая принимает текущий элемент массива (`item`) и возвращает его удвоенное значение.
- Результат работы `map` — новый массив, где каждый элемент умножен на 2.

Пример:

```
let arr = [1, 2, 3, 4, 5];
let newArr = arr.map(item => item * 2);
console.log(newArr); // [2, 4, 6, 8, 10]
```

4. Метод `filter` для фильтрации массива

```
let filteredArr = arr.filter(item => item > 2);
```

Комментарий:

- `filter` — это метод массива, который создаёт новый массив, включая только те элементы, которые удовлетворяют условию.
- `item => item > 2` — стрелочная функция, которая возвращает `true`, если элемент больше 2, и `false` в противном случае.
- Результат работы `filter` — новый массив, содержащий только элементы, которые больше 2.

Пример:

```
let arr = [1, 2, 3, 4, 5];  
let filteredArr = arr.filter(item => item > 2);  
console.log(filteredArr); // [3, 4, 5]
```

5. Метод `reduce` для агрегации массива

```
let sum = arr.reduce((acc, item) => acc + item, 0);
```

Комментарий:

- `reduce` — это метод массива, который используется для агрегации (суммирования, накопления и т.д.) элементов массива.
- `(acc, item) => acc + item` — стрелочная функция, которая принимает два аргумента:
 - `acc` (аккумулятор) — текущее накопленное значение.
 - `item` — текущий элемент массива.
- `0` — начальное значение аккумулятора.
- На каждой итерации `acc` увеличивается на значение `item`.
- Результат работы `reduce` — сумма всех элементов массива.

Пример:

```
let arr = [1, 2, 3, 4, 5];
let sum = arr.reduce((acc, item) => acc + item, 0);
console.log(sum); // 15
```

Итог:

- **Цикл for** — классический способ перебора массива по индексам.
 - **forEach** — удобный метод для перебора элементов массива с доступом к индексу.
 - **map** — создаёт новый массив, применяя функцию к каждому элементу.
 - **filter** — создаёт новый массив, включая только элементы, удовлетворяющие условию.
 - **reduce** — используется для агрегации массива (например, суммирования).
-

Практическое задание:

1. Создайте массив чисел и используйте цикл `for` для вывода каждого элемента.
2. Используйте метод `forEach`, чтобы вывести элементы массива и их индексы.
3. Создайте новый массив, где каждый элемент исходного массива умножен на 3, используя метод `map`.
4. Отфильтруйте массив, оставив только элементы больше 10, используя метод `filter`.
5. Найдите сумму всех элементов массива, используя метод `reduce`.

Пример:

```
let numbers = [5, 10, 15, 20];

// 1. Цикл for
for (let i = 0; i < numbers.length; i++) {
    console.log(numbers[i]);
}

// 2. forEach
numbers.forEach((item, index) => {
    console.log(item, index);
});

// 3. map
```

```
let multiplied = numbers.map(item => item * 3);
console.log(multiplied); // [15, 30, 45, 60]

// 4. filter
let filtered = numbers.filter(item => item > 10);
console.log(filtered); // [15, 20]

// 5. reduce
let sum = numbers.reduce((acc, item) => acc + item, 0);
console.log(sum); // 50
```

3. Объекты в JavaScript (45 минут)

- Объекты — это коллекции пар ключ-значение.
- Создание объекта:

```
let obj = {
  name: "John",
  age: 30,
  greet: function() {
    console.log("Hello!");
  }
};
```

- Доступ к свойствам:

```
console.log(obj.name); // John
console.log(obj["age"]); // 30
```

- Итерация по объектам:

```
for (let key in obj) {
  console.log(key, obj[key]);
}
```

4. Работа с DOM (1 час)

- DOM (Document Object Model) — это представление HTML-документа в виде дерева объектов.
- Поиск элементов:

```
let element = document.getElementById("myId");
let elements = document.getElementsByClassName("myClass");
let elementsByTag = document.getElementsByTagName("div");
let elementBySelector = document.querySelector(".myClass");
let elementsBySelectorAll = document.querySelectorAll("div");
```

- Изменение содержимого:

```
element.innerHTML = "<strong>New content</strong>";
element.textContent = "New text";
```

- Работа с атрибутами:

```
element.setAttribute("data-id", "123");
let id = element.getAttribute("data-id");
element.removeAttribute("data-id");
```

- События:

```
element.addEventListener("click", function() {
    console.log("Element clicked!");
});
```

5. Асинхронность в JavaScript (45 минут)

- Асинхронность позволяет выполнять код без блокировки основного потока.
- `setTimeout` и `setInterval`:

```
setTimeout(() => {
    console.log("Hello after 2 seconds");
}, 2000);

setInterval(() => {
    console.log("Hello every 1 second");
}, 1000);
```

- Промисы:

```
let promise = new Promise((resolve, reject) => {
    setTimeout(() => {
```

```
        resolve("Success!");
    }, 2000);
});

promise.then(result => {
    console.log(result); // Success!
}).catch(error => {
    console.error(error);
});
```

- `async/await`:

```
async function fetchData() {
    try {
        let response = await fetch("https://api.example.com/data");
        let data = await response.json();
        console.log(data);
    } catch (error) {
        console.error(error);
    }
}
```

Давайте подробно разберём каждый из примеров кода, связанных с асинхронностью в JavaScript.

1. Асинхронность и её важность

Теория:

Асинхронность позволяет выполнять код без блокировки основного потока выполнения. Это особенно важно в JavaScript, который является однопоточным языком. Без асинхронности выполнение длительных операций (например, запросов к серверу или таймеров) блокировало бы весь интерфейс или выполнение других задач.

2. `setTimeout` и `setInterval`

`setTimeout`

```
setTimeout(() => {
    console.log("Hello after 2 seconds");
}, 2000);
```

Комментарий:

- `setTimeout` — это функция, которая позволяет выполнить код через определённое время.
- Принимает два аргумента:
 1. Функция, которая будет выполнена (в данном случае стрелочная функция `() => { ... }`).
 2. Время в миллисекундах, через которое функция будет выполнена (2000 мс = 2 секунды).
- После истечения указанного времени функция выполнится **один раз**.

Пример работы:

```
console.log("Start");
setTimeout(() => {
    console.log("Hello after 2 seconds");
}, 2000);
console.log("End");

// Вывод:
// Start
// End
// Hello after 2 seconds (через 2 секунды)
```

setInterval

```
setInterval(() => {
    console.log("Hello every 1 second");
}, 1000);
```

Комментарий:

- `setInterval` — это функция, которая позволяет выполнять код **периодически** с заданным интервалом.
- Принимает два аргумента:

1. Функция, которая будет выполнена (в данном случае стрелочная функция `() => { ... }`).
 2. Интервал в миллисекундах, через который функция будет повторяться (1000 мс = 1 секунда).
- Функция будет выполняться **бесконечно**, пока не будет остановлена с помощью `clearInterval`.

Пример работы:

```
let counter = 0;
let intervalId = setInterval(() => {
  counter++;
  console.log("Hello every 1 second");
  if (counter === 5) {
    clearInterval(intervalId); // Остановить интервал после 5 выполнений
  }
}, 1000);

// Вывод:
// Hello every 1 second (каждую секунду, 5 раз)
```

3. Промисы (Promises)

```
let promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Success!");
  }, 2000);
});

promise.then(result => {
  console.log(result); // Success!
}).catch(error => {
  console.error(error);
});
```

Комментарий:

- Промис — это объект, который представляет результат асинхронной операции. Он может находиться в одном из трёх состояний:
 1. **Ожидание (pending)** — начальное состояние.

2. **Выполнено (fulfilled)** — операция завершена успешно.

3. **Отклонено (rejected)** — операция завершена с ошибкой.

- `new Promise((resolve, reject) => { ... })` — создание нового промиса.
 - `resolve` — функция, которая вызывается при успешном завершении операции.
 - `reject` — функция, которая вызывается при ошибке.
- Внутри промиса используется `setTimeout`, чтобы имитировать асинхронную операцию, которая завершится через 2 секунды.
- `promise.then(result => { ... })` — метод `then` вызывается, когда промис успешно завершён (`resolve`).
 - `result` — значение, переданное в `resolve` (в данном случае `"Success!"`).
- `.catch(error => { ... })` — метод `catch` вызывается, если промис завершён с ошибкой (`reject`).
 - `error` — значение, переданное в `reject`.

Пример работы:

```
console.log("Start");

let promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Success!");
  }, 2000);
});

promise.then(result => {
  console.log(result); // Success!
}).catch(error => {
  console.error(error);
});

console.log("End");

// Вывод:
// Start
// End
// Success! (через 2 секунды)
```

4. `async/await`

```
async function fetchData() {
  try {
    let response = await fetch("https://api.example.com/data");
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Комментарий:

- `async/await` — это синтаксический сахар для работы с промисами, который делает код более читаемым и похожим на синхронный.
- `async function` — объявление асинхронной функции. Внутри такой функции можно использовать ключевое слово `await`.
- `await` — приостанавливает выполнение функции до тех пор, пока промис не будет выполнен.
 - Если промис завершён успешно, `await` возвращает его результат.
 - Если промис завершён с ошибкой, выбрасывается исключение, которое можно обработать с помощью `try/catch`.
- В данном примере:
 1. `fetch("https://api.example.com/data")` — асинхронный запрос к API.
 2. `await fetch(...)` — ожидание завершения запроса.
 3. `response.json()` — преобразование ответа в JSON (тоже асинхронная операция).
 4. `await response.json()` — ожидание завершения преобразования.
 5. Результат выводится в консоль.
- Если происходит ошибка (например, сетевой сбой), она перехватывается блоком `catch`.

Пример работы:

```
async function fetchData() {
  try {
    console.log("Fetching data...");
    let response = await
fetch("https://jsonplaceholder.typicode.com/posts/1");
    let data = await response.json();
    console.log("Data received:", data);
  } catch (error) {
```

```
        console.error("Error:", error);
    }
}

fetchData();

// Вывод:
// Fetching data...
// Data received: { userId: 1, id: 1, title: "...", body: "..." } (через
некоторое время)
```

Итог:

- `setTimeout` — выполнение кода через определённое время.
- `setInterval` — периодическое выполнение кода.
- **Промисы** — удобный способ работы с асинхронными операциями.
- `async/await` — современный синтаксис для работы с промисами, который делает код более читаемым.

Практическое задание:

1. Используйте `setTimeout`, чтобы вывести сообщение через 3 секунды.
2. Создайте интервал, который выводит сообщение каждую секунду, и остановите его через 5 секунд.
3. Напишите промис, который завершается успешно через 2 секунды, и обработайте его результат.
4. Используйте `async/await`, чтобы сделать запрос к API (например, `https://jsonplaceholder.typicode.com/posts`) и вывести результат.

Пример:

```
// 1. setTimeout
setTimeout(() => {
    console.log("Hello after 3 seconds");
}, 3000);

// 2. setInterval
let counter = 0;
```

```

let intervalId = setInterval(() => {
    counter++;
    console.log("Interval tick", counter);
    if (counter === 5) {
        clearInterval(intervalId);
    }
}, 1000);

// 3. Промис
let promise = new Promise((resolve, reject) => {
    setTimeout(() => {
        resolve("Promise resolved!");
    }, 2000);
});

promise.then(result => {
    console.log(result);
});

// 4. async/await
async function fetchPosts() {
    try {
        let response = await
fetch("https://jsonplaceholder.typicode.com/posts");
        let posts = await response.json();
        console.log(posts);
    } catch (error) {
        console.error("Error fetching posts:", error);
    }
}

fetchPosts();

```

6. Заключение и вопросы (20 минут)

- Подведение итогов:
 - Массивы и объекты — основные структуры данных.
 - DOM — основа взаимодействия с веб-страницами.
 - Асинхронность — важная часть современного JavaScript.
- Ответы на вопросы.
- Рекомендации:
 - Практиковаться на небольших проектах.
 - Изучить дополнительные темы: модули, замыкания, прототипы, классы.

Домашнее задание:

1. Создать массив чисел и выполнить его обработку с использованием методов `map` , `filter` , `reduce` .
2. Создать объект, представляющий книгу, и добавить методы для изменения и получения свойств.
3. Создать простую веб-страницу с кнопкой, которая изменяет текст при клике.
4. Реализовать асинхронный запрос к API (например, используя `fetch`) и вывести результат на страницу.

Следующая лекция:

- Модули в JavaScript.
- Замыкания и области видимости.
- Классы и ООП в JavaScript.